

Activity Report: AI Factory Benchmarking Framework

EU Challenge - Student Challenge 2025-2026

Team 6 EUMaster4HPC

Alessandro Ruzza, Valerio Grillo, Michał Sterzel, Davide Villani

Supervised by: Dr. Farouk Mansouri (LuxProvide)

Institution: LuxProvide MeluXina Supercomputer

February 2026

Abstract

This report provides an overview of the work conducted by Team 6 over the past four months during the Student Challenge 2025-2026. It details the development of the **AI Factory Benchmarking Framework** for the MeluXina supercomputer, the organization of the team, the evaluation of results, and the key technical and professional lessons learned during the project.

Contents

1	Introduction and Work Delivered	2
1.1	Project Overview	2
1.2	Key Deliverables	2
1.3	Evaluation of Results	2
1.3.1	1. Performance and Acceleration	2
1.3.2	2. Execution Efficiency	2
1.3.3	3. Monitoring Reliability	3
2	Work Organization	3
2.1	Project Timeline	3
2.2	Team Coordination and Task Distribution	3
3	Lessons Learned	4
3.1	Technical Gains	4
3.2	Professional and Collaborative Gains	4
4	Conclusion	4

1 Introduction and Work Delivered

1.1 Project Overview

The European Union is actively investing in “AI Factories”—High-Performance Computing (HPC) facilities designed to accelerate AI research and deployment. As these facilities come online, there is a critical need for robust evaluation methodologies to benchmark and optimize AI infrastructure components.

Our team delivered a comprehensive **AI Factory Benchmarking Framework** specifically designed for the MeluXina supercomputer. The framework evaluates the end-to-end performance of critical AI components, moving beyond simple raw compute benchmarks to assess the full AI pipeline.

1.2 Key Deliverables

Over the course of the challenge, we delivered a modular, Python-based system comprising the following core capabilities:

- **Unified Benchmarking Tool:** A CLI-driven tool capable of benchmarking 10+ distinct AI services, including LLM inference servers (Ollama, vLLM, Triton), Vector Databases (ChromaDB, Faiss, Weaviate), Relational Databases (PostgreSQL), and Object Storage (MinIO).
- **HPC Integration:** Full integration with the SLURM workload manager for automated job orchestration and Apptainer (Singularity) for secure, reproducible container deployments on MeluXina.
- **Production-Grade Monitoring:** A specialized monitoring stack using Prometheus and Grafana deployed directly on compute nodes, enabling real-time visualization.
- **Reproducibility Engine:** A YAML-based “Recipe” system that ensures every benchmark scenario allows for exact replication of experiments.

1.3 Evaluation of Results

We evaluated the success of our framework based on three criteria: **Performance, Scalability, and Reliability**.

1.3.1 1. Performance and Acceleration

We successfully validated the framework’s ability to measure hardware acceleration. Our benchmarks on MeluXina demonstrated significant performance gains when utilizing NVIDIA GPUs compared to CPU-only baselines:

- **LLM Inference (vLLM):** Achieved a **10-20x** performance improvement in throughput and latency reduction.
- **Vector Search (Faiss):** demonstrated **5-10x** faster indexing and retrieval speeds.
- **Throughput:** Validated high-throughput scenarios with vector databases handling 1000+ requests per second.

1.3.2 2. Execution Efficiency

We implemented a parallel execution mode using Python’s `ThreadPoolExecutor`. Our evaluation showed that this approach yielded up to **10x faster** benchmark execution for I/O-bound tasks compared to sequential modes, significantly reducing the time required for comprehensive system profiling.

1.3.3 3. Monitoring Reliability

The monitoring architecture was evaluated by its ability to capture ephemeral job metrics. By utilizing a Pushgateway on the compute node, we ensured 100% metric capture even for short-lived benchmark jobs, providing granular visibility into P50, P95, and P99 latency distributions.

2 Work Organization

2.1 Project Timeline

The project was conducted over a four-month period, structured into four distinct phases as outlined in our challenge overview:

1. Month 1: Onboarding & Design (Phase 1)

- Introduction to the MeluXina supercomputer architecture.
- Exploration of existing tools and definition of the design specification.
- Selection of the technology stack (Python 3.12, Apptainer, SLURM).

2. Month 2: Prototyping (Phase 2)

- Development of the core “Main Orchestrator” and “Middleware” components.
- Creation of initial Apptainer definition files for AI services.
- Setup of local development environments to simulate HPC constraints.

3. Month 3: Implementation & Integration (Phase 2-3)

- Integration of the Monitoring Stack (Prometheus/Grafana) on compute nodes.
- Development of the SLURM script generation logic.
- Implementation of the “Unified Client Factory” to handle diverse protocols (HTTP, gRPC).

4. Month 4: Evaluation & Documentation (Phase 3-4)

- Deployment at scale on MeluXina for performance measurements.
- Refinement of YAML recipes for reproducibility.
- Drafting the final Technical Report, Poster, and preparing for the defense.

2.2 Team Coordination and Task Distribution

To maximize efficiency, we divided responsibilities based on component modularity while maintaining weekly synchronization:

- **Orchestration & Core Logic:** One workstream focused on the **Main Orchestrator** and **Middleware**, ensuring the CLI correctly parsed YAML recipes and managed the application lifecycle.
- **HPC Integration:** A dedicated focus was placed on the **Server Manager**, handling the translation of service requirements into valid SLURM batch scripts and managing Apptainer container lifecycles.
- **Service Implementation:** We distributed the workload of implementing specific service clients (e.g., one member focused on LLM/vLLM integration, another on Vector/SQL Databases) to ensure parallel development of service support.

- **Monitoring Infrastructure:** A specific task force worked on the observability pipeline, ensuring Prometheus and Grafana could run securely on compute nodes without external internet access.

3 Lessons Learned

Participating in this challenge provided unique insights into the intersection of AI and High-Performance Computing.

3.1 Technical Gains

- **Dynamic Resource Discovery:** A major technical hurdle was allowing clients to find services in a distributed environment. We developed a robust method for dynamic endpoint discovery by parsing SLURM job outputs, a technique essential for loosely coupled HPC microservices.
- **Advanced Monitoring Architectures:** We moved beyond standard monitoring to implement a compute-node-resident stack. We learned how to use SSH tunneling to securely visualize internal HPC metrics on local machines.
- **Declarative Infrastructure:** By implementing the YAML recipe system, we gained a deep appreciation for Infrastructure-as-Code principles, seeing firsthand how declarative configurations simplify reproducibility in scientific experiments.

3.2 Professional and Collaborative Gains

- **DevOps in Research:** The challenge bridged the gap between software development and system operations. We learned that successful AI benchmarking isn't just about the model, but about the orchestration, storage I/O, and network latency that support it.
- **Distributed Collaboration:** Working as a team on a modular codebase required strict interface definitions. We learned to design components (like the `Client Factory`) that could be developed independently yet integrated seamlessly.

4 Conclusion

Over the past four months, Team 6 has successfully architected and deployed a production-ready **AI Factory Benchmarking Framework**. We met our primary objectives of delivering a unified, reproducible, and observable toolset for the MeluXina supercomputer.

The project not only resulted in a functional software artifact capable of benchmarking 10+ AI services but also provided the team with invaluable experience in modern HPC workflows. We believe this framework establishes a solid foundation for future work, including multi-node distributed benchmarking and ML training workload analysis.